Projet Approximations: Méthodes et théorie

Mateus Auza Cruz

Decembre 2024

1 Exercice 1

1.1

Rappels :

Le polynôme de Newton est défini par la formule :

$$N(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots$$

ou encore :

$$N_k(x) = \sum_{i=0}^{k} a_i \prod_{j=0}^{i} (x - x_j),$$

où les a_i sont les coefficients obtenus via les différences divisées.

On veut trouver une formule explicite pour ces différences divisées. De plus, on a $N(x_k) = f(x_k)$.

Ainsi,

$$N(x_0) = a_0 = f(x_0) = f[x_0] \quad \text{(notation)}.$$

$$N(x_1) = a_0 + a_1(x_1 - x_0) \implies a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

Posons cette définition de différence divisée (non démontrée, mais utilisée dans cette démonstration et dans l'exercice 1.3) :

$$\mathbf{a_k} = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}.$$

Théorème d'interpolation de Newton :

Ce polynôme N est égal au polynôme d'interpolation de Lagrange associé aux $k + 1$ points, c'est-à-dire à l'unique polynôme L de degré inférieur ou égal à k vérifiant :

$$\forall i \in \{0, \ldots, k\}, \quad L(x_i) = y_i.$$

Démonstration :

Partie 1 - Montrons que le coefficient de degré k de L est égal aux différences divisées de Newton par récurrence. Pour $k = 1$, le résultat est immédiat :

$$f[x_0] = a_0 = N(x_0) = L(x_0).$$

Supposons que ceci est vrai pour k points. Montrons-le pour $k + 1$ points. Ainsi,

$$L(x) = \sum_{j=0}^{n} \left(f(x_j) \prod_{\substack{i=0 \\ i \neq j}}^{j} \frac{(x - x_i)}{(x_j - x_i)} \right).$$

On peut aussi écrire $L(x)$ comme :

$$L(x) = \frac{(x - x_0)Q(x) - (x - x_k)P(x)}{x_k - x_0},$$

où $P(x)$ est le polynôme d'interpolation associé aux k premiers points (indices 0 à $k - 1$) et $Q(x)$ est celui associé aux k derniers points (indices 1 à k).

On remarque que $Q(x) \cdot (x - x_0) = f[x_1, \ldots, x_k]$ et que $P(x) \cdot (x - x_k) = f[x_0, \ldots, x_{k-1}]$.

Alors,

$$L(x) = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0} = f[x_0, \ldots, x_k] = a_k.$$

Ceci est la différence divisée posée ci-dessus. Donc, le coefficient en x^k du polynôme de Lagrange est celui de Newton.

Partie 2 - Montrons maintenant que $L = N$. Pour un point, on a :

$$f(x_0) = a_0 = N(x_0) = L(x_0).$$

Supposons que notre hypothèse de récurrence est vraie pour k points. $L - N$ est de degré inférieur ou égal à k, nul en $x_0, \ldots, x_k$, et son coefficient de degré k est égal à celui de L, soit $a_k = f[x_0, \ldots, x_k]$.

Ainsi,

$$L(x) = N(x) + a_k(x - x_0) \cdots (x - x_{k-1}).$$

Par notre hypothèse de récurrence, ceci est égal à $N(x)$.

■

Puisque les coefficients de degré k de $L(x)$ sont égaux à a_k par la partie 1 de la démonstration, alors pour calculer a_k, il nous suffit de calculer les coefficients en x^k de $L(x)$.

1.0.1 Calculons-les :

$$N(x) = L(x) = \sum_{j=0}^{n} \left(f(x_j) \prod_{\substack{i=0 \\ i \neq j}}^{j} \frac{(x - x_i)}{(x_j - x_i)} \right).$$

On a :

$$\prod_{\substack{i=0 \\ i \neq j}}^{j} \frac{(x - x_i)}{(x_j - x_i)} = \frac{x^n + x^{n-1} + \cdots + x^i + \cdots + c}{\prod_{\substack{i=0 \\ i \neq j}}^{j} (x_j - x_i)}.$$

On met x^i en évidence dans le numérateur et on remarque que le coefficient de degré i est donné par :

$$\frac{1}{\prod_{\substack{i=0 \\ i \neq j}}^{j} (x_j - x_i)}.$$

Donc,

$$a_i = \sum_{j=0}^{n} \frac{f(x_j)}{\prod_{\substack{i=0 \\ i \neq j}}^{j} (x_j - x_i)}.$$

1.2

Partie 1: Prouver que si $x_0, x_1 \to x^*$, alors $a_k = f^{(1)}(x^*)$

On a que les nœuds $x_0, x_1 \to x^*$ et que $a_1 = f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$. Posons $h = x_1 - x_0$. Si $x_0, x_1 \to x^*$, alors $h \to 0$.

On peut réécrire a_1 comme :

$$a_1 = \frac{f(x_0 + h) - f(x_0)}{h}.$$

Puisque $x_0 \to x^*$, alors :

$$a_1 = \lim_{x_0 \to x^*} \frac{f(x_0 + h) - f(x_0)}{h} = \frac{f(x^* + h) - f(x^*)}{h}.$$

Alors :

$$\lim_{h \to 0} a_1 = \lim_{h \to 0} \frac{f(x^* + h) - f(x^*)}{h}.$$

Mais cela correspond à la définition de $f'(x^*)$. Donc a_1 converge vers $f'(x^*)$.

Partie 2: Si $x_0, \ldots, x_k \to x^*$, alors $a_k = \frac{f^{(k)}(x^*)}{k!}$

On a que le polynôme de Newton peut s'écrire comme :

$$N_k(x) = \sum_{i=0}^{k} a_i \prod_{j=0}^{i} (x - x_j),$$

Si tous les nœuds x_j convergent vers x^*, alors on peut simplifier l'expression du polynôme de Newton comme :

$$N_k(x) = \sum_{i=0}^{k} a_i (x - x^*)^i.$$

Lemme : Les coefficients a_i d'une série du type $N_k(x) := \sum_{i=0}^{k} a_i (x - x^*)^i$, où $x^* \in \mathbb{R}$,(mais on peut généraliser ce résultat pour $x^* = z \in \mathbb{C}$, voir cours d'analyse complexe), peuvent s'écrire comme :

$$\lim_{x \to x^*} \left(N_k^{(i)}(x) \right) = i!\, a_i.$$

Preuve : On peut exprimer la série $N_k(x)$ comme :

$$N_k(x) = a_0 + a_1(x - x^*) + a_2(x - x^*)^2 + \ldots + a_i(x - x^*)^i + \ldots + a_k(x - x^*)^k.$$

Donc,

$$N_k^{(i)}(x) = 0 + 0 + \ldots + i! a_i + (i+1)! a_{i+1}(x - x^*) + \ldots + \frac{k!}{(k-i)!} a_k (x - x^*)^{k-i}.$$

Si on fait :

$$\lim_{x \to x^*} \left(N_k^{(i)}(x) \right) = N_k^{(i)}(x^*) = i!\, a_i + (i+1)! a_{i+1} \cdot 0 + \ldots + \frac{k!}{(k-i)!} a_k \cdot 0^{k-i} = i!\, a_i.$$

Donc,

$$a_i = \frac{N_k^{(i)}(x^*)}{i!}.$$

■

Par le théorème d'interpolation de Lagrange, on a que $N_k(x^*) = f_k(x^*)$, donc $N_k^{(i)}(x^*) = f^{(i)}(x^*)$.

En simplifiant, on obtient :

$$a_i = \frac{f^{(i)}(x^*)}{i!}.$$

■

Partie 3: Põlynome de Newton et de Taylor

On a, par le rappel de 1.1, que le pôlynome de Newton peut s'ècrire comme:

$$N_k(x) = \sum_{i=0}^{k} a_i \prod_{j=0}^{i} (x - x_j),$$

où les a_i sont les coefficients obtenus via les différences divisées.

Par la Partie 2, on a que si $x_0, \ldots, x_k \to x^*$, alors $a_k = \frac{f^{(k)}(x^*)}{k!}$. Donc, on peut dire que le polynôme de Newton (polynôme interpolateur) avec $(n+1)$ nœuds confluent converge vers le polynôme :

$$N_k(x) = \sum_{i=0}^{k} \frac{f^{(i)}(x^*)}{i!} \prod_{j=0}^{i} (x - x^*)^i.$$

Mais on remarque que ceci est le polynôme de Taylor de degré n évalué en x^*.

1.3

Définition des nœuds équidistants dans un intervalle $[-t, t]$

Pour cette première partie, je me restreins uniquement au cas où $t = \frac{1}{8}$ (car c'est la valeur intermédiaire entre $t = \frac{1}{16}$ et $t = \frac{1}{4}$), afin d'expliquer pédagogiquement mon raisonnement. Dans le cas où $t = \frac{1}{16}$ ou $t = \frac{1}{4}$, on obtient les mêmes résultats.

```
T = 1/8  # Je d finis ceci dans le scope global pour ne pas avoir
       rappeler constamment que l'on traite le cas o   T = 1/8
N_nodes=7 #Nombre de noueds   quidistants
```

Pour générer les nœuds équidistants, on utilise la commande `np.linspace` :

```
import numpy as np
def equidistant_nodes(t, n):
    return np.linspace(-t, t, n)
```

Maintenant qu'on a les nœuds, on veut calculer le polynôme de Newton pour $\sin(x)$. Pour cela, on a besoin de calculer les différences divisées (déjà expliquées à l'exercice 1.1). Puisque les différences divisées sont définies récursivement, nous avons besoin de toutes les différences divisées.

Diferences divisées:

Exemple : Posons trois nœuds x_0, x_1, et x_2. On a alors :

$$a_0 = f[x_0], \quad a_1 = f[x_0, x_1], \quad a_2 = f[x_0, x_1, x_2]$$

dans le polynôme de Newton. Mais on calcule également d'autres différences divisées, par exemple $f[x_1, x_2]$. Nous avons besoin de $f[x_1, x_2]$ pour calculer

$f[x_0, x_1, x_2]$, en utilisant la formule récursive des différences divisées posée à l'exercice 1.

Ainsi, il est préférable d'organiser ces calculs dans une matrice. On pose dans la première colonne de la matrice les $y_{\text{data}} = f(x_{\text{data}})$ et dans la première ligne les coefficients de Newton recherchés. La deuxième colonne contient toutes les différences divisées d'ordre 1 (calculées à partir de deux nœuds), la troisième colonne contient toutes les différences divisées d'ordre 2 (calculées à partir de trois nœuds), et ainsi de suite.

Pour notre exemple, la matrice des différences divisées est donnée par :

$$Dd = \begin{pmatrix} f[x_0] & f[x_0, x_1] & f[x_0, x_1, x_2] \\ f[x_1] & f[x_1, x_2] & 0 \\ f[x_2] & 0 & 0 \end{pmatrix}$$

On remarque que :

$$a_2 = f[x0, x1, x2] = Dd[0][2] = \frac{Dd[1][1] - Dd[0][1]}{x_{\text{data}}[2] - x_{\text{data}}[0]}.$$

Cas général : Pour le cas général, la formule s'écrit:

$$Dd[i][j] = \frac{Dd[i+1][j-1] - Dd[i][j-1]}{x_{\text{data}}[i+j] - x_{\text{data}}[i]}.$$

Code:

```
def div_diff(x_data, y_data,t):
    n = len(x_data)
    Dd = np.zeros((n, n))  # Matrice des differences divisees
    Dd[:, 0] = y_data

    for j in range(1, n):
        for i in range(n - j):
            Dd[i][j] = (Dd[i + 1][j - 1] - Dd[i][j - 1]) / (x_data[
                i + j] - x_data[i])
    return Dd
```

Posons aussi la variable globale: (on va l'utiliser tout au long du code)

```
X_data= equidistant_nodes(T, N_nodes)
```

Calcul du polynôme de Newton

Code:

```
import sympy as sp

def Newton_polynomial(x_data, Dd,t):
    x = sp.symbols("x")
    n = len(Dd[0])
    polynome = Dd[0][0]
    eta = 1  # Base du polynome de Newton

    for i in range(1, n):
```

```
        eta *= (x - x_data[i - 1])
        polynome += Dd[0][i] * eta
    return polynome
```

Le polynôme obtenu est en mode symbolique. Pour le rendre numérique (et donc le tracer), on utilise `sp.lambdify` :

```
def Interpolation(x,t):
    x_data = X_data  #Variable defini audessous
    y_data = np.sin(x_data)
    Dd = div_diff(x_data, y_data,t)
    poly_lamb = sp.lambdify(sp.symbols("x"), Newton_polynomial(
        x_data, Dd,t), "numpy")
    return poly_lamb(x)
```

Tracé des graphiques

Traçons maintenant le polynôme de Newton et la fonction $\sin(x)$ sur l'intervalle $[-1, 1]$:

```
import matplotlib.pyplot as plt
def sin_et_newton(value,N,t):
    plt.title("Graphes de sin(x) et le polynome de Newton")
    plt.plot(np.linspace(-value, value, N), Interpolation(np.
        linspace(-value, value, N),t),
    label=f"Polynome interpolateur de sin(x) avec t={t}")
    plt.plot(np.linspace(-value, value, N), np.sin(np.linspace(-
        value, value, N)), label="sin(x)", color="red")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Ilustration 1:

```
print(sin_et_newton(1,100,T)) #T est defini globalement
```

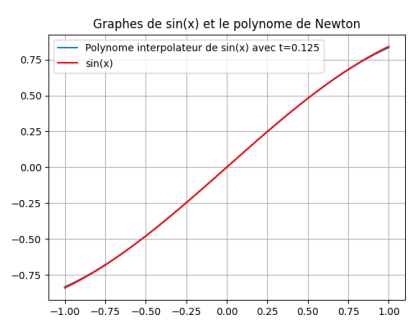

Figure 1

Ilustration 2:

```
print(sin_et_newton(5,100,T))
```

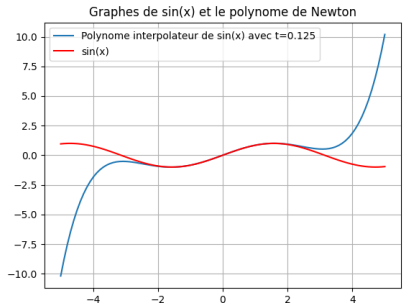

Figure 2

Observations On remarque que l'interpolation est précise autour de 0, mais diverge en dehors de l'intervalle $[-t = -1/8, t = 1/8]$. Ceci est attendu, car le polynôme interpole uniquement dans cet intervalle.

Comparaison avec le polynôme de Taylor

Le polynôme de Taylor autour de 0 peut être calculé symboliquement avec `sp.series` :

```
def Taylor(order):
    a = sp.symbols("a")
    f = sp.sin(a)
    return f.series(a, 0, order).removeO()
```

Pour le transformer en polynôme numérique (en utilisant la methode sp.lambidfy) :

```
def Taylor_Interpolation(a,order):
    Taylor_lamb = sp.lambdify(sp.symbols("a"), Taylor(order), "numpy")
    return Taylor_lamb(a)
```

Plot de tout les graphes: Traçons $\sin(x)$, le polynôme de Newton, et le polynôme de Taylor :

```
def Sin_Newton_Taylor(order, t, value,N):
    plt.title("Graphes de sin(x), polynome de Newton et polynome de taylor autour de 0")
    plt.plot(np.linspace(-value, value, N), Interpolation(np.linspace(-value, value, N),t), label=f"Polynome interpolateur de sin(x) avec t={t}")
    plt.plot(np.linspace(-value, value, N), np.sin(np.linspace(-value, value, N)), label="sin(x)", color="red")
```

```
    plt.plot(np.linspace(-value, value, N), Taylor_Interpolation(np.linspace(-value, value, N), order), label=f"polynome de taylor de sin(x) autour 0 de degree {order}", color="green")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Ilustration 1:

```
print(Sin_Newton_Taylor(order=7,t=T, value=5,N=100))
```

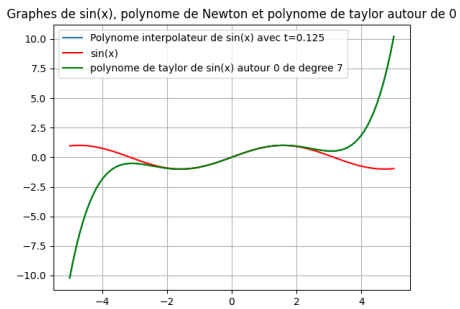

Figure 3

Ilustration 2:

```
print(Sin_Newton_Taylor(order=8,t=T, value=5,N=100))
```

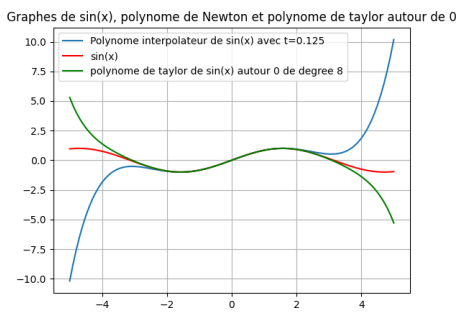

Figure 4

Observations: On remarque à partir de ces 2 ilustrations que le polynôme interpolateur de Newton et le polynôme de Taylor s'intersectent, quand le nombre de nœuds (N_{nodes}) est égal à l'ordre du polynôme de Taylor (Order) quand

l'ordre du polynôme de Taylor est égale au degrée du polynôme de Newton. (On ne montre pas ce résultat, mais le correcteur peut changer N_{nodes} et Order pour observer cette affirmation). Donc, on peut soupçonner que le polynôme de Newton pour des nœuds près de 0 est en effet le polynôme de Taylor autour de 0.

Tracés des graphes pour différents *order* et t

Les tracés sont obtenus avec le code suivant :

```python
def tout_les_plots(orders,t_values,N):
    fig, axes = plt.subplots(len(orders), len(t_values), figsize=(15, 10), sharex=True, sharey=True)
    for i,order in enumerate(orders):
        for j,t in enumerate(t_values):
            ax = axes[i, j]
            ax.plot(np.linspace(-5,5,N),Interpolation(np.linspace(-5,5,N),t), label=f"Polynome interpolateur de sin(x) avec t={t}")
            ax.plot(np.linspace(-5,5,N),np.sin(np.linspace(-5,5,N)), label="sin(x)", color="red")
            ax.plot(np.linspace(-5,5,N),Taylor_Interpolation(np.linspace(-5,5,N),order), label=f"polynome de taylor de sin(x) autour 0 de degree 7", color="green")
            ax.set_title(f"order={order}, t={t}")
            ax.legend()

    fig.suptitle("Graphes de sin(x), Polynome de Newton, et du polynome de Taylor", fontsize=16)
    fig.supxlabel("x")
    fig.supylabel("y")
    plt.tight_layout(rect=[0, 0, 1, 0.95])  # Adjust space to fit the title
    plt.show()
```

Ilustration:

```python
print(tout_les_plots(orders=[5,10,30],t_values=[1/4,1/8,1/16],N=100))
```

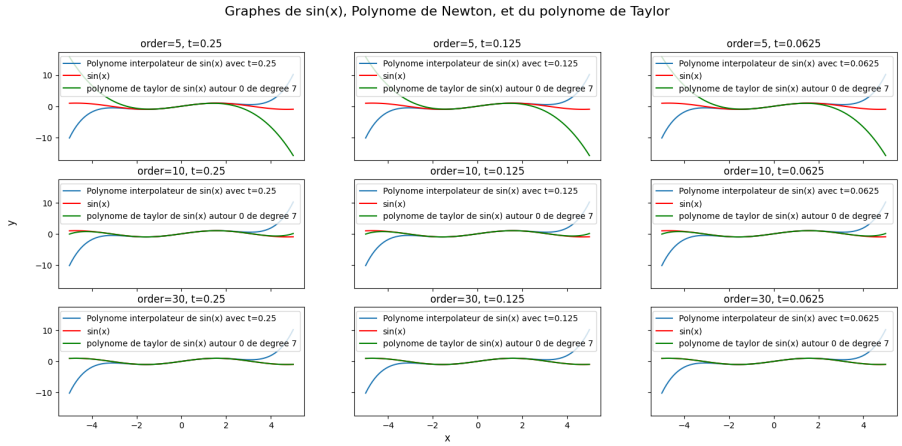

Figure 5

Comparaison de l'erreur de Lagrange pour $\mathbf{t = \frac{1}{8}}$

Calcul de l'erreur théorique **Remarque :** Comme mentionné précédemment, pour $t = \frac{1}{16}$ et $t = \frac{1}{4}$, on obtient les mêmes résultats.

On peut écrire la formule de l'erreur théorique comme suit. Pour calculer l'erreur théorique, on veut calculer le terme de droite :

$$|\sin(x) - \text{Newton's Polynomial}| \leq \frac{1}{(n+1)!} \sup |f^{(n+1)}(x)| \cdot W_{n+1}(x),$$

avec $W_{n+1}(x) = \prod_{i=0}^{n}(x - x_i)$.

Puisque $f(x) = \sin(x)$, alors $\forall n \in \mathbb{N}$, on a $\sup |f^{(n+1)}(x)| \leq 1$.

Cela simplifie la formule en :

$$|\sin(x) - \text{Newton's Polynomial}| \leq \frac{W_{n+1}(x)}{(n+1)!}.$$

Calculons le terme de droite en Python :

```python
from math import factorial

def theorical_error_bound(x_data, degree_newton):
    x = sp.symbols("x")
    W = 1
    for i in range(len(x_data)):
        W *= (x - x_data[i])
    bound = abs(W) / factorial(degree_newton+1)
    return bound
```

Par la méthode sp.lambdify, on obtient la version numérique :

```python
def Num_bound(x, x_data, degree_newton):
    bound_lamb = sp.lambdify(sp.symbols("x"), theorical_error_bound
        (x_data, degree_newton), "numpy")
    return bound_lamb(x)
```

Par nos observations des graphiques, l'erreur serait très petite pour des valeurs autour de 0, mais grande pour des valeurs éloignées de 0.

Exemple:

```
value1=0.001
print(Num_bound(value1, X_data, 6))
>>3.7347707575308675e-14
value2=7
print(Num_bound(value2, X_data, 6))
>>163.32034666351822
```

Donc, nos observations étaient correctes.

Comparaison avec l'erreur actuelle Calculons le terme de gauche de la formule du reste de Lagrange en Python:

```
def actual_error_bound(x_data, y_data,t):
    x = sp.symbols("x")
    Dd = div_diff(x_data, y_data,t)  # I need y_data for the divided differences
    error = abs(sp.sin(x) - Newton_polynomial(x_data, Dd,t))
    return error

def Num_actual_bound(x, x_data, y_data,t):
    actual_lamb = sp.lambdify(sp.symbols("x"), actual_error_bound(x_data, y_data,t), "numpy")
    return actual_lamb(x)
```

Exemple:

```
print(Num_actual_bound(value1, X_data, np.sin(X_data),T))
>>3.7329948160413906e-14

print(Num_actual_bound(value2, X_data, np.sin(X_data),T))
>>89.15365761808133
```

On peut observer que les résultats numériques sont corrects et que l'erreur theorique est superieur à l'erreur actuelle. Et donc, on a mit en pratique l'inequation du reste de Lagrange.

2 Exercice 2: Courbes de Bézier et les splines

Exercice 2.1: Une courbe de Bézier pour n points

Pour écrire la formule générale d'une courbe de Bézier, on peut soit l'écrire sous forme récursive, soit sous forme non-récursive.

Formule récursive: La formule récursive des courbes de Bézier est élégante mais peu efficace, et devient très difficile à calculer pour un ordinateur dès qu'on dépasse 4 points. L'autre méthode, bien que moins élégante, est beaucoup plus efficace. (J'utilise la formule non-recursive pour le projet).

*Notation: Soient $P_0, \ldots, P_n$ des points dans un espace affine, et $0 \leq t \leq 1$.

- **Cas de base:**

$$B(t; P_0, P_1) = (1-t)P_0 + tP_1 \quad \text{(un segment de droite)}$$

- **Récursion:**

$$B(t; P_0, \ldots, P_n) = (1-t)B(t; P_0, \ldots, P_{n-1}) + tB(t; P_1, \ldots, P_n)$$

Code Python pour la formule récursive:

```python
import numpy as np
import sympy as sp

def Bezier_curve_rec(P):
    t = sp.symbols("t")
    if len(P) == 2:
        return (1 - t) * P[0] + t * P[1]
    B_rec = (1 - t) * Bezier_curve_rec(P[:-1]) + t *
        Bezier_curve_rec(P[1:])
    return B_rec
```

Formule non-récursive: Avec les polynômes de Bernstein, la formule devient:

$$B(t) = \sum_{k=0}^{n} C(n,k)(1-t)^{n-k} t^k P_k$$

Remarque importante: Dans cet exercice, on se limite aux courbes de Bézier en dimension 2, car c'est le cas le plus simple.

Code Python pour la version non-récursive:

```python
import sympy as sp
import numpy as np
def Bezier_curve_x(P):
    t = sp.symbols("t")
    n = len(P) - 1
    B_x = 0
    for k in range(n + 1):
        B_x += sp.binomial(n, k) * (1 - t)**(n - k) * t**k * P[k
            ][0]
    return B_x

def Bezier_curve_y(P):
    t = sp.symbols("t")
    n = len(P) - 1
    B_y = 0
    for k in range(n + 1):
        B_y += sp.binomial(n, k) * (1 - t)**(n - k) * t**k * P[k
            ][1]
    return B_y
```

Plot de notre courbe de Bézier On transforme la courbe de Bézier symbolique en numérique par la méthode sp.lambdify :

```
def plot_Bezier_curve_x(P,t):
    Bezier_lamb_x= sp.lambdify(sp.symbols("t"),Bezier_curve_x(P),"numpy")
    return Bezier_lamb_x(t)

def plot_Bezier_curve_y(P,t):
    Bezier_lamb_y= sp.lambdify(sp.symbols("t"),Bezier_curve_y(P),"numpy")
    return Bezier_lamb_y(t)
```

On utilise les fonctions Python précédentes pour calculer et tracer la courbe.

```
import matplotlib.pyplot as plt

def plot_spline(t_values,P):
    Points = np.array(P)
    B_x_values = plot_Bezier_curve_x(P,t_values)
    B_y_values = plot_Bezier_curve_y(P,t_values)

    plt.title(f"Courbe de B zier avec {len(Points)} points")
    plt.plot(B_x_values, B_y_values, color="blue", label="Courbe de B zier")
    plt.scatter(Points[:, 0], Points[:, 1], color="red", marker="o", label="Points")
    plt.plot(Points[:, 0], Points[:, 1], linestyle="--", color="black")
    plt.legend()
    plt.xlabel("x")
    plt.ylabel("y")
    plt.axis("equal")
    plt.grid(True)
    plt.show()
```

Ilustration:

```
print(plot_spline(t_values = np.linspace(0, 1, 100),P= [(0, 2), (2, 5), (8, 5), (5, 4), (0, 3), (10, 6)]))
```

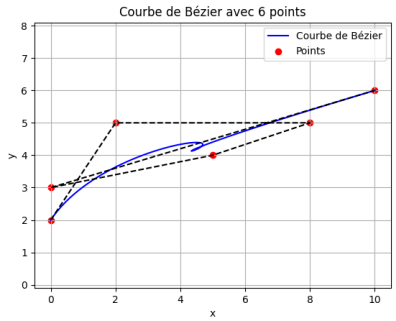

Figure 6

Exercice 2.2: Deux courbes de Bézier Quadratiques

Pour cet exercice, j'ai repris le code de l'exercice 2.1 et ajouté un point **Q**. Le point P sera l'ensemble des 3 points de la premiere spline et Q sera l'ensemble des 3 ppoints de la deuxieme spline.

Code:

```python
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt


def Bezier_curve_x(P):
    t = sp.symbols("t")
    n = len(P) - 1
    B_x = 0
    for k in range(n + 1):
        B_x += sp.binomial(n, k) * (1 - t)**(n - k) * t**k * P[k
            ][0]
    return B_x

def Bezier_curve_y(P):
    t = sp.symbols("t")
    n = len(P) - 1
    B_y = 0
    for k in range(n + 1):
        B_y += sp.binomial(n, k) * (1 - t)**(n - k) * t**k * P[k
            ][1]
    return B_y

def plot_Bezier_curve_x(P,t):
    Bezier_lamb_x= sp.lambdify(sp.symbols("t"),Bezier_curve_x(P),"
        numpy")
    return Bezier_lamb_x(t)

def plot_Bezier_curve_y(P,t):
    Bezier_lamb_y= sp.lambdify(sp.symbols("t"),Bezier_curve_y(P),"
        numpy")
    return Bezier_lamb_y(t)

def finale(P,Q,t_values):
    if len(P) != len(Q) or len(Q) != 3:  # V rifie que ce sont des
         courbes de B zier quadratiques
        raise ValueError("Bezier␣curve␣must␣be␣quadratic")
    if np.any(P[-1] != Q[0]):  # Assure la continuit  de la spline
        . Sinon on ne peut pas coller les 2 splines
        raise ValueError("Spline␣must␣have␣knots␣well␣defined")


    P=np.array(P)
    Q=np.array(Q)

    #Plot des courbes de B zier pour les points P et Q

    B_x_P = plot_Bezier_curve_x(P, t_values)
    B_y_P = plot_Bezier_curve_y(P, t_values)

```

```python
    plt.title(f"Spline construite avec deux courbes de B zier")
    plt.plot(B_x_P, B_y_P, color="blue", label="Courbe de B zier pour P")
    plt.scatter(P[:, 0], P[:, 1], color="red", marker="o", label="P")
    plt.plot(P[:, 0], P[:, 1], linestyle="--", color="black")

    #
    B_x_Q = plot_Bezier_curve_x(Q, t_values)
    B_y_Q = plot_Bezier_curve_y(Q, t_values)

    plt.plot(B_x_Q, B_y_Q, color="green", label="Courbe de B zier pour Q")
    plt.scatter(Q[:, 0], Q[:, 1], color="yellow", marker="x", label="Q")
    plt.plot(Q[:, 0], Q[:, 1], linestyle="--", color="gray")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Ilsutration:

```python
P = [(0, 2), (5, 7), (2, 5)]
Q = [(2, 5), (4, 8), (6, 7)]
t_values=np.linspace(0, 1, 100)
print(finale(P,Q,t_values))
```

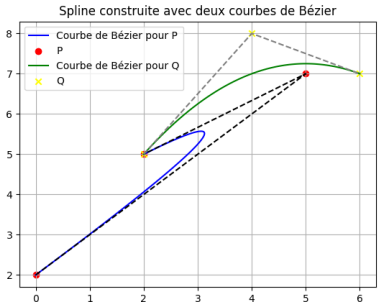

Figure 7

Degrés de Liberté : Non, la spline n'est pas déterminée, comme vu en cours. Elle possède des degrés de liberté liés au degré de la spline et au nombre de points qui connectent les courbes de Bézier.

Pour calculer cela, il faut résoudre le système d'équations associé, en déterminant le nombre de contraintes dans le système et le nombre d'inconnues. On en déduit que cette courbe de Bézier possède 3 degrés de liberté.

Exo 2.3: Generalisation pour n courbes de Bézier Quadratiques

Au lieu de définir plusieurs fonctions distinctes pour chaque courbe de Bézier (comme dans l'exercice 2.2), j'ai utilisé une boucle for qui traite directement chaque segment, ce qui est beaucoup plus efficace.

Code:

```python
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np

def Bezier_curve_x(P):
    t= sp.symbols("t")
    n=len(P)-1
    B_x=0

    for k in range(n+1):
        B_x+= sp.binomial(n,k)*(1-t)**(n-k) * t**k *P[k][0]

    return B_x

def Bezier_curve_y(P):
    t= sp.symbols("t")
    n=len(P)-1
    B_y=0

    for k in range(n+1):
        B_y+= sp.binomial(n,k)*(1-t)**(n-k) * t**k *P[k][1]

    return B_y

def plot_Bezier_curve_x(P,t):
    Bezier_lamb_x= sp.lambdify(sp.symbols("t"),Bezier_curve_x(P),"numpy")
    return Bezier_lamb_x(t)

def plot_Bezier_curve_y(P,t):
    Bezier_lamb_y= sp.lambdify(sp.symbols("t"),Bezier_curve_y(P),"numpy")
    return Bezier_lamb_y(t)


def finale(Points,t_values):
    plt.figure()
    plt.title(f"Spline construite avec {len(Points)} courbes de Bezier quadratiques")
    for i,P in enumerate(Points):
        P=np.array(P)
        if len(P)!=3:
            raise ValueError("Bezier curve must be quadratic")

        if i!=0 and i!= len(Points)-1:
            if np.any(Points[i][-1]!= Points[i+1][0]):
                raise ValueError("Spline must have knots well defined")

        B_x_P= plot_Bezier_curve_x(P,t_values)
```

```
        B_y_P= plot_Bezier_curve_y(P,t_values)

        plt.plot(B_x_P, B_y_P, label= f"Courbe de Bezier segment {i+1}")
        plt.scatter(P[:,0],P[:,1],marker="o", label=f"Points du segment {i+1}")
        plt.plot(P[:,0],P[:,1],linestyle= "--")

    plt.legend()
    plt.xlabel("x")
    plt.ylabel("y")
    plt.axis("equal")
    plt.grid(True)
    plt.show()
```

Ilustration:

```
Points=[[(0,2),(5,7),(2,5)],
        [(2,5),(4,8),(6,7)],
        [(6,7),(9,8),(-2,7)],
        [(-2,7),(4,1),(-6,-7)]]
t_values=np.linspace(0,1,100)
print(finale(Points,t_values))
```

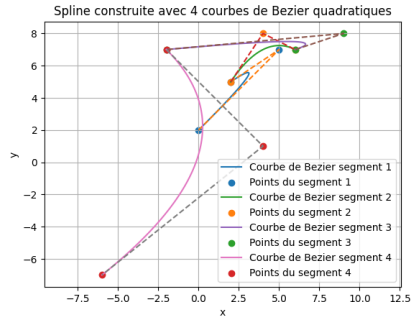

Figure 8

Degrées de liberté: Non, la spline n'est pas déterminée. Elle possède des degrés de liberté liés au degré de la spline et au nombre de points qui connectent les courbes de Bézier.

Pour calculer cela, il faut résoudre le système d'équations associé, en déterminant le nombre de contraintes dans le système et le nombre d'inconnues. On en déduit que cette courbe de Bézier possède **n+1** degrés de liberté, où n est le nombre de splines qui composent la courbe de Bézier.

3 Exercice 3: Polynômes Orthogonaux de Legendre

3.1: Coefficients de Legendre

Polynôme de Legendre

Premièrement, on importe les packages nécessaires :

```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
from sympy import sqrt
```

Pour calculer les polynômes de Legendre via le produit scalaire, on doit utiliser la procédure de Gram-Schmidt qui orthogonalise une certaine famille de vecteurs.

Dans ce cas, on travaille dans l'espace L^2. Prenons une base non orthonormée de cet espace. Le cas le plus simple est la base des polynômes :

$$P_0 = 1, \quad P_1 = x, \quad P_2 = x^2, \ldots, \quad P_n = x^n$$

Pour calculer les polynômes de Legendre via le produit scalaire, on utilise la formule de récurrence suivante :

$$\begin{aligned} e_0 &= P_0, \\ k_i &:= P_i - \sum_{k=0}^{i-1} \frac{\langle P_i, e_k \rangle \cdot e_k}{\|e_k\|_{L^2}^2}, \\ e_i &= \frac{k_i}{\|k_i\|_{L^2}}, \end{aligned}$$

En Python, cela donne :

Code:

```
x = sp.symbols("x")

def base_poly(n):
    l = []
    for i in range(n+1):
        l.append(x**i)
    return l

def scalar_product(f, g): # Le produit scalaire en L2
    x = sp.symbols("x")
    return sp.integrate(f * g, (x, -1, 1))

def norm(f): # La norme L2
    return sqrt(scalar_product(f, f))

def legendre_poly_normal(n):
    base = base_poly(n)
```

```
    orthog_poly = []  # Liste des polyn mes de Legendre
    orthog_poly.append(base[0] / norm(base[0])) # Premier    lment

    # R currence
    for i in range(1, len(base)):
        poly = base[i]
        for k in range(i):
            poly -= scalar_product(base[i], orthog_poly[k]) *
                orthog_poly[k] / (norm(orthog_poly[k])**2)
        orthog_poly.append(poly / norm(poly))
    return orthog_poly
```

Exemple:

```
print(legendre_poly_normal(4))
>>[sqrt(2)/2, sqrt(6)*x/2, 3*sqrt(10)*(x**2 - 1/3)/4, 5*sqrt(14)*(x
    **3 - 3*x/5)/4, 105*sqrt(2)*(x**4 - 6*x**2/7 + 3/35)/16] # Les
    4 premiers polynomes orthogonaux nornmalisees en (1,1)
```

Three term recurrence

Pour calculer la three term recurrence, on a vu dans le cours théorique que si on pose $P_{-1} = 0$, on a :

$$xP_n = a_{n+1}P_{n+1} + b_nP_n + a_nP_{n-1}$$

où P_n est le polynôme orthogonal de degré n, $a_n = \langle xP_n, P_{n-1}\rangle$, et $b_n = \langle xP_n, P_n\rangle$.

En Python, cela donne :

```
def three_term_relation(n):
    x = sp.symbols("x")
    poly = legendre_poly_normal(n+1) # Important: Pas les standards

    P_n = poly[n]
    P_n_plus = poly[n+1]
    P_n_minus = poly[n-1]

    a_n_plus = scalar_product(x * P_n, P_n_plus)
    b_n = scalar_product(x * P_n, P_n)
    a_n_minus = scalar_product(x * P_n, P_n_minus)

    three_rec = a_n_plus * P_n_plus + b_n * P_n + a_n_minus *
        P_n_minus

    return sp.simplify(three_rec) == sp.simplify(x * P_n)
```

3.0.1 Relation de recurrence bonus:

La relation de récurrence des polynômes de Legendre peut s'écrire comme :

$$(n+1)P_{n+1} = (2n+1)xP_n - nP_{n-1}$$

avec $P_0 = 1$ et $P_1 = x$, où n est le degré du polynôme de Legendre.

Démonstration: La preuve est faite à partir des fonctions génératrices énoncées au cours. Voici la preuve : **Lien vers la démonstration sur math.libretexts.org**

Code:

```
def recurrence_relation(n): # here I calculate the normlalized legendre polynomials
    x=sp.symbols("x")
    P0=1
    P1=x
    if n==-1:
        return 0
    if n==0:
        return P0
    if n==1:
        return P1

    P_prev=P0
    P_curr=P1

    for k in range(1,n):
        P_new=((2*k+1)*x*P_curr-k*P_prev)/(k+1)

        P_prev=P_curr
        P_curr=P_new

    Normal=P_curr

    return sp.expand(Normal)
```

3.2: Plotter les polynomes de Legendre et leurs racines:

Cette question utilise le même code que 3.1

Pour passer des polynômes de Legendre symboliques aux polynômes de Legendre numériques, on utilise la méthode `lambdify` :

```
def Legendre_interpolation(x,n):
    Legendre_lamb = sp.lambdify(sp.symbols("x"), recurrence_relation(n), "numpy")
    return Legendre_lamb(x)
```

Pour tracer les graphes et les racines :

```
def plot_Legendre(n_data):
    for i,n in enumerate(n_data):
        roots=sp.solve(recurrence_relation(n),x)
        numeric_roots = [float(root.evalf()) for root in roots]

        plt.title("P lynomes de Legendre")
        plt.plot(np.linspace(-1,1,100),Legendre_interpolation(np.linspace(-1,1,100),n), label=f"P lynome orthogonal de Legendre de degr {n}")
        plt.scatter(numeric_roots, [0] * len(numeric_roots))
    plt.legend()
    plt.grid(True)
    plt.show()
```

Exemple:

```
n_data=[1,2,5,10]
print(plot_Legendre(n_data))
```

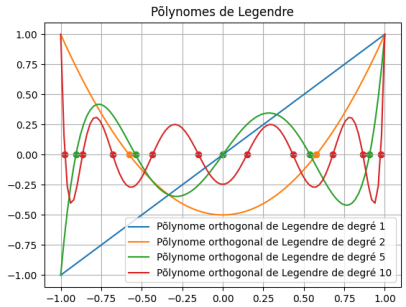

Figure 9

Exo 4 - Quadrature de Legendre et Chebychev

Conseil: Pour le code de cet exercice je trouve que c'est mieux de mettre 2 codes separements, un seulement pour Legendre et l'autre pour Chebychev

4.1 - Implémentation des règles de quadrature pour Legendre et Chebychev sur $[-1, 1]$

Le but de cet exercice est de calculer les règles de quadrature d'une fonction. On veut calculer :

$$\int f(x)w(x)\,dx = \sum_{i=0}^{k} w_i f(x_i)$$

où x_i est la racine du polynôme orthogonal et w_i les poids.

Règle de quadrature de Legendre

Produit scalaire et norme : Dans l'espace L^2 pour $w(x) = 1$, on implémente :

```
#######CODE LEGENDRE-QUADRATURE##################
import numpy as np
import sympy as sp
from sympy import sqrt

def scalar_product(f, g):
    x = sp.symbols("x")
    return sp.integrate((f * g), (x, -1, 1))
```

```python

def norm(f):
    return sqrt(scalar_product(f, f))
```

Relation de récurrence : Calcul des polynômes de Legendre normalisés en $[-1, 1]$:

```python
def recurrence_relation(n):  # Polyn mes de Legendre normalis s
    x = sp.symbols("x")
    P0 = 1
    P1 = x

    if n == -1:
        return 0
    if n == 0:
        return P0
    if n == 1:
        return P1

    P_prev = P0
    P_curr = P1

    for k in range(1, n):
        P_new = ((2 * k + 1) * x * P_curr - k * P_prev) / (k + 1)
        P_prev = P_curr
        P_curr = P_new

    return sp.expand(P_curr)
```

Racines: Calcul des racines des polynômes de Legendre (x_i).

```python
def roots_Legendre(n):
    x = sp.symbols("x")
    roots = sp.solve(recurrence_relation(n), x)
    return sorted([float(root.evalf()) for root in roots])

def derivative_Legendre(n):
    x = sp.symbols("x")
    return sp.diff(recurrence_relation(n), x)

def Legendre_interpolation(x,n): # Methode sp.lambdify vu audessous
    Legendre_lamb=sp.lambdify(sp.symbols("x"),derivative_Legendre(n
        ),"numpy")
    return Legendre_lamb(x)
```

Poids: Maintenant, on veut calculer les poids de la fonction de Legendre, c'est-à-dire les w_i.

Dans le cas de Legendre, on a que la formule peut être écrite comme suit :

$$w_i = \frac{2}{(1 - x_i^2) \cdot P_n'(x_i)}.$$

Démonstration: On sait que la formule générale des poids peut être écrite comme :

$$w_i = \int_a^b l_i(x) w(x)\, dx,$$

où $l_i(x)$ est le polynôme de Lagrange donné par :

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

On peut simplifier comme suit :

$$\prod_{\substack{j=0 \\ j \neq i}}^{n} (x - x_j) = \frac{P_n(x)}{K_n \cdot (x - x_i)},$$

où K_n est le coefficient du terme de degré x^n dans $P_n(x)$.

De plus, on a :

$$\prod_{\substack{j=0 \\ j \neq i}}^{n} (x_i - x_j) = \frac{P_n'(x_i)}{K_n}.$$

Ainsi,

$$w_i = \int_a^b \frac{P_n(x)}{K_n \cdot (x - x_i)} \cdot \frac{K_n}{P_n'(x_i)} \cdot w(x)\, dx = \int_a^b \frac{P_n(x)}{P_n'(x_i) \cdot (x - x_i)} \cdot w(x)\, dx.$$

Maintenant, simplifions cela pour $w(x) = 1$ et sur l'intervalle $[-1, 1]$:

$$w_i = \int_{-1}^{1} \frac{P_n(x)}{P_n'(x_i) \cdot (x - x_i)}\, dx.$$

Posons :

$$P_i(x) = \frac{P_n(x)}{P_n'(x_i) \cdot (x - x_i)}.$$

On sait que $P_i(x_j) = \delta_{ij}$, le produit de Kronecker (1 si $i = j$, 0 sinon).

Ainsi :

$$w_i = \int_{-1}^{1} P_i(x)\, dx.$$

On a alors :

$$P_n'(x_i) \cdot w_i = \int_{-1}^{1} P_n'(x) \cdot P_i(x)\, dx.$$

En intégrant par parties avec $u' = P_n'(x)$ et $v = P_i(x)$, on obtient :

$$P_n'(x_i) \cdot w_i = \Big[P_n(x) \cdot P_i(x) \Big]_{-1}^{1} - \int_{-1}^{1} P_n(x) \cdot P_i'(x)\, dx.$$

On remarque que :

$$\int_{-1}^{1} P_n(x) \cdot P_i'(x)\, dx$$

est un intégral symétrique. Comme $P_n(x)$ est impair ou pair selon n, et $P_i'(x)$ est de même parité opposée, cet intégral est nul.

Ainsi, il reste :

$$P_n'(x_i) \cdot w_i = \left[\frac{P_n(x)^2}{P_n'(x_i) \cdot (x - x_i)}\right]_{-1}^{1}.$$

En évaluant cette expression, on trouve :

$$P_n'(x_i) \cdot w_i = \frac{1}{(1 - x_i^2) \cdot P_n'(x_i)} + \frac{1}{(1 - x_i^2) \cdot P_n'(x_i)} = \frac{2}{(1 - x_i^2) \cdot P_n'(x_i)}.$$

Donc,

$$w_i = \frac{2}{(1 - x_i^2) \cdot P_n'(x_i)^2}. \quad \blacksquare$$

Code:

```
def weights_Legendre(n):
    roots = roots_Legendre(n)
    weights = []
    for root in roots:
        weight = 2 / ((1 - root**2) * Legendre_interpolation(root,n
            )**2)
        weights.append(weight)
    return weights
```

Approximation finale : La quadrature de Legendre est donnée par :

```
def final(function, n):
    roots = roots_Legendre(n)
    weights = weights_Legendre(n)
    integral = 0
    for i in range(len(roots)):
        integral += weights[i] * function(roots[i])
    return integral
##################################################
```

Règle de quadrature de Chebychev

Produit scalaire et norme : Pour $w(x) = \frac{1}{\sqrt{1-x^2}}$:

```
##########CODE CHEBYCHEV-QUADRATURE##########################3
import numpy as np
import sympy as sp
from sympy import sqrt
def scalar_product(f, g):
    x = sp.symbols("x")
    return sp.integrate((f * g) / sqrt(1 - x**2), (x, -1, 1))
```

```python

def norm(f):
    return sqrt(scalar_product(f, f))
```

Polynômes de Chebychev : Pour cet exercice, je vais utiliser uniquement les polynômes de première espèce, mais pour les polynômes de deuxième espèce, on obtient les mêmes conclusions. On a que les polynomes de Chevychev de premiere espece sont definis comme: $T_n(\cos\theta) = \cos(n\theta).\forall\theta \in \mathbb{R}$ On peut exprimer ces polynomes ápd de la formule de recurrence suivante:

$$\begin{aligned} T_0(x) &= 1, \\ T_1(x) &= x, \\ T_{n+1}(x) &= 2x\,T_n(x) - T_{n-1}(x). \end{aligned}$$

Preuve :

Pour $n = 0$ et $n = 1$, on a :

$$T_0(x) = \cos(0 \cdot \theta) = 1, \quad T_1(x) = \cos(\theta) = x.$$

$$T_{n+1}(x) + T_{n-1}(x) = \cos((n+1)\theta) + \cos((n-1)\theta)$$

On a:

$$= \cos(n\theta)\cos(\theta) - \sin(n\theta)\sin(\theta) + \cos(n\theta)\cos(\theta) + \sin(n\theta)\sin(\theta)$$

$$= 2\cos(\theta)\cos(n\theta) = 2xT_n(x).$$

En réarrangeant ces formules, on obtient :

$$T_{n+1}(x) + T_{n-1}(x) = 2T_n(x)x.$$

■

Code:

```python
def recurrence_relation(n):
    x = sp.symbols("x")
    T0 = 1
    T1 = x

    if n == 0:
        return T0
    if n == 1:
        return T1

    T_prev = T0
    T_curr = T1

    for k in range(1, n):
        T_new = 2 * x * T_curr - T_prev
        T_prev = T_curr
        T_curr = T_new

    return sp.expand(T_curr)
```

Racines:

```python
def roots_chebychev(n):
    x = sp.symbols("x")
    roots = sp.solve(recurrence_relation(n), x)
    return [float(root.evalf()) for root in roots]
```

Poids: On a vu dans le syllabus que la formule des poids de Chebychev pouvait être écrite comme :

$$w_i = \frac{\pi}{n+1} \quad \text{(indépendamment de la racine).}$$

Demonstration: Nous avons dans ce cas (voir l'Exercice 4.9.13) :

$$P_{n+1}(x) = 2^{-n}T_{n+1}(x), \quad n \in \mathbb{N}, \quad P_0 = T_0,$$

et :

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \quad T_0(x) = 1, \quad T_1(x) = x,$$

d'où :

$$P_0(x) = 1 \quad \text{et} \quad P_{n+1}(x) = xP_n(x) - \frac{1}{4}P_{n-1}(x),$$

avec :

$$P_1(x) = x, \quad P_2(x) = x^2 - \frac{1}{2}, \quad n \in \mathbb{N} \setminus \{0, 1\}.$$

Par le Théorème 5.4.1, nous avons que :

$$\varphi_0(x) = 0 \quad \text{et que} \quad \varphi_{n+1}(x) = x\varphi_n(x) - \frac{1}{4}\varphi_{n-1}(x),$$

avec :

$$\varphi_1(x) = \pi, \quad \varphi_2(x) = \pi x.$$

En posant $\varphi_{n+1}(x) = \pi 2^n U_n(x)$, nous obtenons :

$$U_{n+1}(x) = 2xU_n(x) - U_{n-1}(x), \quad U_0(x) = 1, \quad U_1(x) = 2x.$$

Ceci est précisément la relation de récurrence pour les polynômes de Chebyshev de deuxième type (voir encore l'Exercice 4.9.13). Nous avons donc que :

$$\varphi_{n+1}(x) = \pi 2^{-n}U_n(x) = \pi 2^{-n}\frac{T'_{n+1}(x)}{n+1},$$

et :

$$A_k = \frac{\varphi_{n+1}(x_k)}{P'_{n+1}(x_k)} = \frac{\pi 2^{-n}T'_{n+1}(x_k)}{(n+1)2^{-n}T'_{n+1}(x_k)} = \frac{\pi}{n+1}.$$

■

Approximation finale: **Code:**

```
def final_CH(function, n):
    roots = roots_chebychev(n)
    integral = 0
    for root in roots:
        integral += (np.pi / (n + 1)) * function(root)
    return integral
#######################################
```

4.2 Comparaison des quadratures de Legendre et Chebychev

Fonctions à observer :

```
################POUR LES 2 CODES###########################
from scipy.integrate import quad
import matplotlib.pyplot as plt
def function1(x):
    return 1 / (1 + 25 * x**2)

def function2(x):
    return abs(x)

def function3(x):
    return np.exp(x)

##################################
```

Comparaison et visualisation : Calcul des différences pour les deux quadratures.

3.0.2 Legendre:

```
################CODE LEGENDRE###########################
def comparison(function):
    differences = []
    nodes = [1, 5, 10, 20]
    for node in nodes:
        result, _ = quad(function, -1, 1)
        approx = final(lambda x: function(x), node)
        differences.append(abs(result - approx))
    return nodes, differences

nodes1, differences1 = comparison(function1)
nodes2, differences2 = comparison(function2)
nodes3, differences3 = comparison(function3)

plt.figure(figsize=(10, 6))
plt.plot(nodes1, differences1, label="Integrand 1 (1 / (1 + 25 x  ))", marker='o')
plt.plot(nodes2, differences2, label="Integrand 2 (|x|)", marker='s')
```

```
plt.plot(nodes3, differences3, label="Integrand 3 (exp(x))", marker
    ='^')
plt.xlabel("Nombre de n uds (n)")
plt.ylabel("Diff rence")
plt.title("Comparaison entre l'int grale et la quadrature")
plt.legend()
plt.grid(True)
plt.show()
#########################################
```

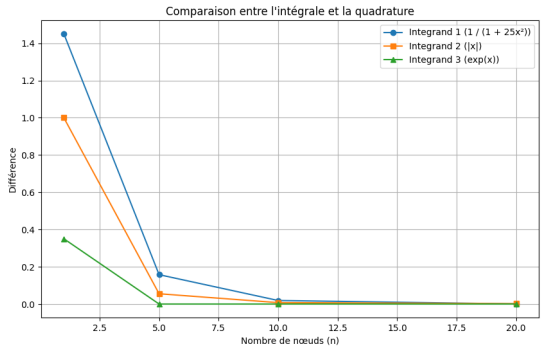

Figure number

Chebychev:

Pour Chebychev, c'est légèrement différent car la fonction poids est

$$w(x) = \frac{1}{\sqrt{1-x^2}}.$$

Donc, on ne veut pas approcher l'intégrale $\int_{-1}^{1} f(x)\,dx$, mais plutôt l'intégrale

$$\int_{-1}^{1} f(x)\frac{1}{\sqrt{1-x^2}}\,dx.$$

Donc, on veut définir :

```
###########CODE CHEBYCHEV-INTEGRAL##################
def chebychev_weight(x):
    return 1 / sqrt(1 - x**2)
```

Puis on définit les intégrandes avec la fonction poids de Chebychev :

```
def integrand1(x):
    return function1(x) * chebychev_weight(x)

def integrand2(x):
    return function2(x) * chebychev_weight(x)
```

```python

def integrand3(x):
    return function3(x) * chebychev_weight(x)
```

Calculons les différences entre la quadrature et l'intégrale :

```python
def comparison(function):
    differences = []
    nodes = [1, 5, 10, 20]
    for node in nodes:
        result, _ = quad(function, -1, 1)
        approx = final(lambda x: function(x) / chebychev_weight(x), node)
        differences.append(abs(result - approx))
    return nodes, differences
```

Calculons les différences pour les 3 intégrandes et faisons les plots:

```python
nodes1, differences1 = comparison(integrand1)
nodes2, differences2 = comparison(integrand2)
nodes3, differences3 = comparison(integrand3)
# Plottons ceci :
plt.figure(figsize=(10, 6))
plt.plot(nodes1, differences1, label="Integrand 1 (1 / (1 + 25 x  ))", marker='o')
plt.plot(nodes2, differences2, label="Integrand 2 (|x|)", marker='s')
plt.plot(nodes3, differences3, label="Integrand 3 (exp(x))", marker='^')
plt.xlabel("Nombre de noeuds (n)")
plt.ylabel("Difference")
plt.title("Comparaison entre l'int grale et la quadrature de Chebychev")
plt.legend()
plt.grid(True)
plt.show()

##########################################
```

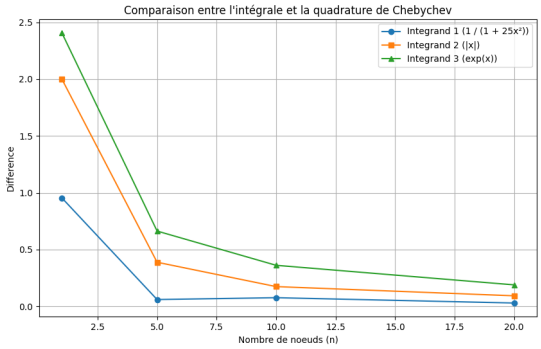

Figure 11

Conclusion

On observe que la quadrature de Chebychev est moins précise en approximant des noeuds. On remarque que la différence avec la quadrature de Legendre est beaucoup plus faible dans tous les cas.

Donc, la quadrature de Legendre est mieux pour approximer des fonctions.

Bibliographie:

- Chebyshev Polynomials (Wikipedia)
- Polynômes de Legendre (Wikipedia en français)
- Gauss-Legendre Quadrature (Wikipedia)
- Spline (Wikipedia)
- B-Spline (Wikipedia)
- Newton Polynomial (Wikipedia)
- Lien vers la démonstration sur math.libretexts.org
- Weighting for Gauss-Legendre Quadrature (Math StackExchange)
- **Syllabus d'approximations**
- Document Moodle UCLouvain
- Splines: Relationship of Knots, Degree, and Degrees of Freedom (Stats StackExchange)